Applied Soft Computing 133 (2023) 109945

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Temporal Convolutional Networks with RNN approach for chaotic time series prediction

Hatice Vildan Dudukcu *, Murat Taskiran, Zehra Gulru Cam Taskiran, Tulay Yildirim

Yildiz Technical University, Department of Electronics and Communication Engineering, 34220 Istanbul, Turkey

ARTICLE INFO

Article history:
Received 12 May 2022
Received in revised form 3 December 2022
Accepted 11 December 2022
Available online 17 December 2022

Keywords:
Time series prediction
Deep neural network
Recurrent neural networks
Temporal convolutional neural network
Chaotic systems
ECG recordings

ABSTRACT

The prediction of chaotic time series, which constitutes many systems in the field of science and engineering, has recently become the focus of attention of researchers. Chaotic time series prediction is making future predictions about these systems using previously observed data for a nonlinear chaotic system with a known initial condition. Chaotic time series prediction can be applied in many fields such as weather forecasting, finance and stock markets. Many disciplines work on solving time series prediction problem, ranging from forecasting weather events days in advance to traders predicting the future of stocks. In recent studies, it has been observed that hybrid deep neural network methods give better performance in solving time series prediction problems and have gained popularity in order to benefit from the advantages of more than one method in solving such problems. In this study, a hybrid deep neural network architecture is proposed for chaotic time series prediction. The used hybrid approach includes both temporal convolutional network to extract low level features from input and recurrent neural network layers such as long short-term memory and gated recurrent units to capture temporal information. Simulations were carried out on nine different chaotic time series dataset which are obtained from Lorenz, Rössler and a Lorenz-like chaotic equation sets, and twenty-one electrocardiogram (ECG) recordings of patients with arrhythmias. In the benchmark study, in which twelve different methods, including classical machine learning, deep neural network and hybrid models were used, the proposed model achieved the best prediction performance with an average root-mean-square error (RMSE) value of 0.0022 for chaotic dataset and 0.0082 for ECG arrhythmia dataset. Performance evaluation metrics show that the proposed hybrid architecture can compete with the models in state-of-the-art studies in chaotic time series prediction.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

The mathematical models of many systems in nature that change over time have dynamic system properties and such systems generate chaotic data and operate on them. Although chaos does not yet have a universally accepted definition, it is in the form of non-linear systems that produce non-periodic output sensitive to initial conditions. Chaotic signals are noise-like signals with a continuous power spectrum. These signals can be found in nature such as fluid flow, heartbeat irregularities, weather and climate data, as well as in many systems with real life patterns such as stock market and road traffic. With the increase in the success of the studies on the prediction of chaotic time series, it will be possible to make precautions and preparations for the future situations of these processes which are difficult to predict. Nowadays, researchers carry out studies to make future predictions on systems containing chaotic time series such as chaotic electroencephalogram (EEG) [1], carrier vibration [2], ozone concentration [3], solar activity [4] and wind speed [5], and necessary system controls can be provided according to these predictions.

The methods used in time series prediction studies in the literature can be divided into three categories as statistical methods, machine learning methods and deep neural networks [6–39] .

Although traditional autoregressive and moving average forecasting methods, which are used in prediction studies, were also used on time series in the early days, they have low estimation performance due to their linear nature. Therefore artificial neural networks (ANN) and machine learning (ML) algorithms have replaced statistical methods in time series prediction problems in the 2000s, due to their high success in dealing with non-linearity [40].

Machine learning and artificial neural networks were widely used in the early 2000s in solving time series prediction problems. Successful results were obtained by combining these methods, which frequently encountered in the literature, with a statistical method [6–11]. In the study carried out by Zhang in 2003, a

* Corresponding author.
E-mail addresses: vdudukcu@yildiz.edu.tr (H.V. Dudukcu), mrttskrn@yildiz.edu.tr (M. Taskiran), zgcam@yildiz.edu.tr (Z.G. Cam Taskiran), tulay@yildiz.edu.tr (T. Yildirim).

https://doi.org/10.1016/j.asoc.2022.109945
1568-4946/© 2022 Elsevier B.V. All rights reserved.

method using a combination of Autoregressive integrated moving average (ARIMA) and ANN was proposed for time series prediction. In experimental studies using three benchmark dataset, the Wolf's sunspot [41], the Canadian lynx [42], and the British pound-US dollar exchange rate dataset, it has been observed that the hybrid method gives more successful results compared to ARIMA and ANN alone [6]. In another study conducted by Jiang and Song in 2010, it was observed that the Nonlinear Autoregressive with eXogenous input (NARX) method gives more successful results in the prediction of exchange rate time series data compared to the backpropagation neural networks and support vector machine (SVM) [11]. As a result of the observations in Kourentzes et al.'s study in 2014, it is inferred that the mode operator should be considered as an alternative to the mean and median operators in estimation applications [10].

Since the beginning of the 2010s, the use of deep neural networks (DNNs), which have been observed to achieve higher performance in solving regression problems, has started to become widespread in literature [12–36]. DNNs, which are broadly used for time series prediction problems, can be grouped under four main network architectures: Elman recurrent neural networks (ERNN) [19–21], long short-term memory (LSTM) [22–31], convolutional neural networks (CNN) [12–18] and temporal convolutional networks (TCN) [32–36]. In the study conducted by Mohammadi et al. in 2017, an improved ERNN was used to provide short term load forecast (STLF) for optimal power system operation. As a result of the experimental studies on the real-world engineering test case, it was observed that the ERNN architecture gives more accurate predictions when compared with 19 different methods [20]. In another study by Ruiz et al. in 2018, ERNN structures were used for energy consumption forecasting in public buildings, and these structures were optimized using a genetic algorithm. They observed that ERNN gives more successful results compared to simpler ML methods, while the complexity of the system increases [21]. In addition to the use of ERNN in time series prediction problems, LSTM, which is another recurrent neural network architecture and contain gates where past input information can be stored, have also been widely used in this field in recent years. Ma et al. in 2015, used LSTM architecture for time series prediction, experimental studies were carried out on sensor data for traffic speed prediction. Their results demonstrated that LSTM gave more successful results compared to ERNN, time-delay neural network (TDNN) and nonlinear autoregressive with exogenous inputs (NARX) neural network [22]. In 2018, Fischer and Krauss used the LSTM method to solve the financial time series predictions problem. In this study, Thomson Reuters dataset was used and the results show that the LSTM method worked with higher accuracy compared to the other methods such as, random forests, standard deep nets, and logistic regression methods [25]. In addition to these studies, Sagheer and Kotb have experimentally demonstrated that LSTM for petroleum production forecasting gives better results than both other recurrent neural network (RNN) and gated recurrent unit (GRU) architectures [28]. CNN structures, which are generally used in 2D data processing and have an important place in solving classification problems, have also been used in the prediction of time series data in recent years, and it has been observed that CNN structures have achieved high performance results in time series prediction problem. In the study, which is one of the examples of this, CNN architecture was used for stock price prediction by Tsantekidis et al. and compared with multilayer neural networks and support vector machine methods in experimental studies performed on benchmark datasets [43] and it was seen that the CNN method was more successful compared to other methods [12]. On another aspect, Kuo and Huang proposed a novel CNN architecture called DeepEnergy for load forecasting in smart grid systems. In the experimental results of the study, DeepEnergy, when compared with other structures such as SVM, MLP and LSTM, was examined with mean absolute percentage error (MAPE) and cumulative variation of root mean square error (CV-RMSE) metrics, and the DeepEnergy method has given better results compared to others [14]. In recent years, researchers have carried out studies on how the hybrid use of different architectures will affect the performance of time series prediction problems. SeriesNet, proposed by Shen et al. in 2020, is an example of these hybrid approaches which includes LSTM and CNN structures. The performance of this method in time series prediction problem has been examined using various datasets. SeriesNet method showed higher forecasting accuracy and much greater stability compared to other methods for the evaluations made root-mean-square error (RMSE), mean absolute error (MAE) and coefficient of determination [18]. In addition to the proposed hybrid use of different models, modified CNN is also among the methods suggested in recent studies. The WaveNet architecture [44,45], which is a newer dilated CNN approach in the literature, was used to solve the financial time series forecasting problem in the study conducted by Borovykh et al. in 2018. The WaveNet method gives more successful results than LSTM in time series predictions [32]. In 2020, Chen et al. using various benchmark datasets, compared the temporal convolutional neural network (TCN), which is another dilated CNN structure, with state-of-the-art methods, and observed that this structure compares favorably for point and probabilistic forecasting [34].

Chaotic time series prediction has became one of the most interesting time series problems for the researchers recently. The reason for this is that many systems in the nature exhibit dynamic behavior and with the prediction of chaotic time series being possible, many phenomena in nature will become predictable. Chandra and Zhang used the cooperative convolution of ERNN method for the prediction of Mackey–Glass, Lorenz and Sunspot time series, which are types of chaotic time series, and with this method, they obtained 0.008 RMSE value for Lorenz time series data [19]. In the study of Guo Yanan et al. in 2020, LSTM was used for the chaotic time series prediction with the data obtained from the Lorenz-63 system, and it was observed that this method gave more successful results compared to other classical NN structures [30]. Sangiorgio and Dercole showed that the accuracy and robustness of the LSTM structures used for the prediction of chaotic time series increases without teacher forcing [31]. In recent years, it has been observed that hybrid structures obtained as a result of combining different architectures, apart from methods involving a uniform cell network, have become widespread in literature. Recent works showed that the combining CNN structures with different neural network modules for the solution of the chaotic time series prediction problem increases the performance [36,46]. Today, with the widespread use of forecasting applications of real life systems, the prediction of chaotic systems has become an important research subject. Hybrid applications have also gained popularity in order to benefit from the advantages of more than one method in solving such problems [47,48].

In this study, a hybrid model formed by combining different neural network layers, the temporal convolutional neural network (TCN) with recurrent neural networks (RNN) is proposed. In simulation studies, using three different data set obtained from Lorenz, Rössler and Lorenz-like equation sets, and also a real life chaotic dataset of 21 ECG recordings of patients with arrhythmia referred to the Arrhythmia Laboratory at Boston's Beth Israel Hospital [49,50], the proposed method and classical ML structures and deep learning methods were compared and it was seen that the proposed model achieved the lowest prediction RMSE value.

In the following section, detailed explanation of the proposed method is given. In Section 3, information about the chaotic

equations used in the data set and the model simulations is explained. Detailed results are also given in this section. Finally, in the Conclusion and Future Work section, the results of proposed model simulations were discussed and the suggestions of the authors about the methods that can be used by the researchers to solve this problem in the future are shared.

2. Methodology

Prediction of chaotic time series has became one of the most interesting topics of researchers recently. Thanks to the prediction of chaotic time series, many systems that exhibit dynamic behavior in nature can be forecasted. The fact that the hybrid use of different neural network structures gives better results for prediction problems compared to classical recurrent neural networks has led current researches to these methods. In this study, TCN with RNN architecture consisting of TCN, LSTM and GRU layers is proposed to solve the chaotic time series prediction problem. This architecture consists of TCN and RNN layers. While TCN contributes to the extraction of the features of changes that occur in a short time in time series, LSTM reveals the features of changes that occur in a long time, and GRU has an effective non-linear fitting capability [48,51]. In this way, it is possible to model all the features that are spread over a wide frequency spectrum. In the model used, TCN extracts one-dimensional space features of time series and gives these feature vectors to RNN. The two networks are trained together, thus obtaining two rounds of feature extraction and two rounds of filtering for data impurities. Since TCN already greatly reduces data impurities, the trailing RNN stage works more effectively and extracts sequential features better. Information on TCN, RNN and a detailed description of our proposed architecture are given in the following subsections.

2.1. Theoretical aspects

2.1.1. Temporal convolutional network

Temporal Convolutional Network (TCN) is a deep learning architecture with dilated, causal 1D convolution layers used for 1D data [38]. This architecture was first used by Lea et al. in 2016 to perform video-based action segmentation and it has dilated CNN layers that calculates low level features. TCN is suitable to use with RNN models that utilizes output from CNN as input to capture temporal information. Dilated CNN layers in TCN can extract features from longer time sequences compared to RNNs by using a large receptive field. The disadvantage of TCN compared to RNN models is that the computational cost is higher and the training time is longer.

TCN architecture given in Fig. 1, consists of a residual block with dilated convolutional layer, normalization, activation, and dropout layers and an optional 1×1 convolution which is used when residual input and output have different dimensions. In dilated convolutional layers, for each hidden vector, the relevant hidden unit value is calculated from the input values and the filter convolution. By using the calculated hidden vector, the vector of the next hidden layer is created with dilated calculations and the output vector is obtained in the last dilated layer. The calculation process of the output vector from the input vector using dilated layers is given in Fig. 2.

2.1.2. Recurrent neural networks

Recurrent neural network (RNN) architectures are deep learning structures that are widely used for prediction in the literature, that can learn and make predictions with inferences from the past information due to the memory connections inside. Although classical RNN structures give successful results in prediction short-term data, they are not useful due to the vanishing gradient problem experienced in problems with long-term data. RNN structures with different memory connections such as gated recurrent units (GRU) and long short-term memory (LSTM) have been proposed to solve this problem. Recently, these two architectures have been used more commonly in prediction problems than classical RNN approaches.

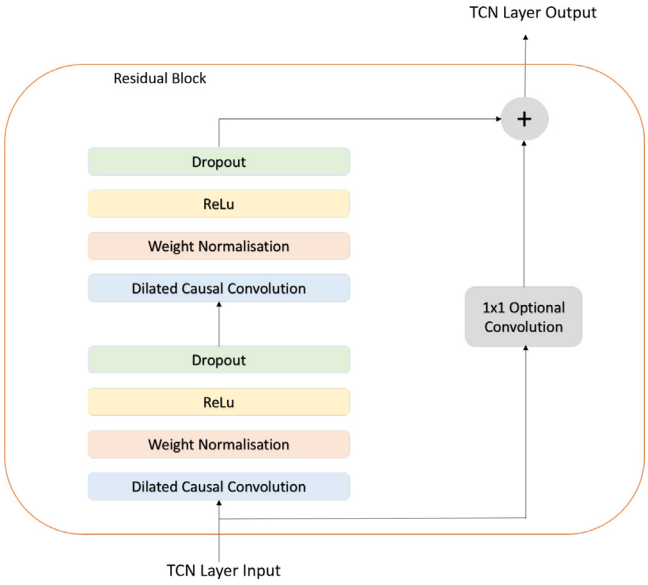


Fig. 1. TCN architecture.

Long Short-Term Memory (LSTM) is a recurrent neural network (RNN) architecture used in the field of deep learning. Unlike standard feed-forward neural networks, LSTM has feedback connections and memory state in it which stores previous outputs of the network [39]. In addition to instant data, it can also process data sequences, and therefore it is frequently used in solving prediction problems. In LSTM deep learning architecture, as shown in Fig. 3, each cell receives a cell state (c_{t-1}), a hidden state (h_{t-1}) and an input (x_t) for time step t, and creates a cell state (c_t) and a hidden state (h_t) at the cell output. Here, hidden states represent the output of the cell, and cell state represents the memory line containing information about previous timesteps.

Gated recurrent unit (GRU) is a simplified and improved type of LSTM structure, with two gates called reset and update gates [52]. The input of a cell at a given time is x_t and the output of the previous cell is h_{t-1}. In GRU structures, a single cell output is created by combining the cell state and hidden state seen in Fig. 3, and past information is transferred to the next times over calculated cell output (h_t).

2.2. Proposed method

In this study a hybrid approach, which includes temporal convolutional neural network with recurrent neural networks, is proposed for time series prediction. When time series prediction literature is examined, it is noteworthy that TCN architecture performs better than LSTM and GRU in solving most prediction problems, and it has been investigated in most studies how many stacked TCN layers give the best results. Due to the structure of the TCN, as the number of layers is increased, the computational load of convolutional layers increase and the model does not always give more successful results. In the proposed hybrid structure for this study, TCN was used as the first DNN layer to extract low-level features in a wide receptive field. The temporal

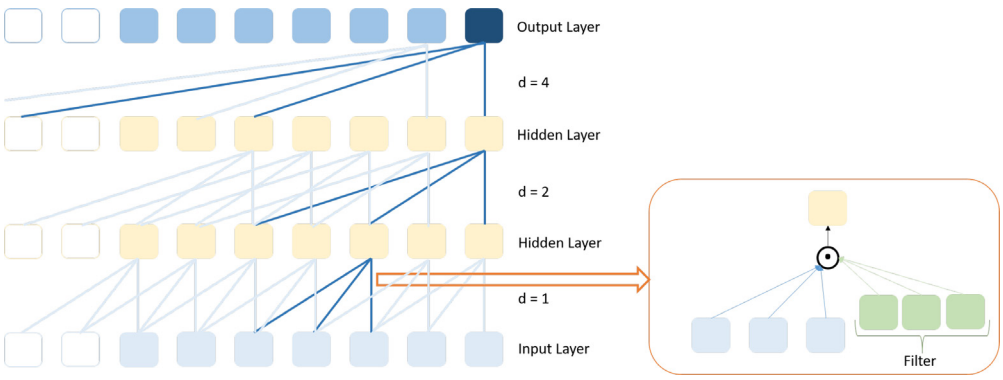


Fig. 2. Dilated convolution with dilation factors d = 1, 2, 4 using filter size 3.

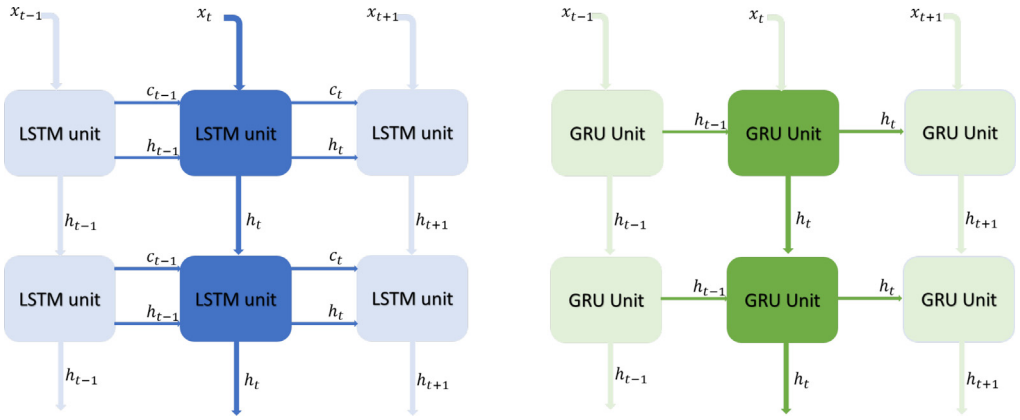


Fig. 3. LSTM and GRU architecture comparison.

information of the features, which is the output from the TCN layer, is processed with the RNN layer in the next step to fully utilize the chaotic time series. Two different hybrid approach are proposed in the study. The first of these approaches, as seen in Fig. 4, is the method that makes predictions using LSTM and a Dense layer at the output of TCN layer, and the second approach includes GRU and Dense Layer with TCN for prediction. Within the scope of this study, the data was converted into a pattern using the TCN layer, and then the pattern was transferred to the future times with memory gates using the RNN layer, thus increasing the prediction performance of the time series.

3. Datasets and model simulations

Model simulations were carried out using nine different chaotic time series which are obtained from chaotic system equation sets and 21 ECG recordings of patients with arrhythmia from the Arrhythmia Laboratory at Boston's Beth Israel Hospital [49,50] in order to test the performance of proposed TCN with RNN. For this purpose, the chaotic time series data is produced from Lorenz, Rössler and a Lorenz-like chaotic system equation sets, respectively. The chaotic time series produced from these three differential equation sets are divided into three subsets as training, validation and test. Among the data subsets, the training set was used for training of deep learning models, the validation set was used to monitor the performance of the training process, and the test set was used to test the performance of the trained model on data that had not been seen before. Furthermore, as a real life example, for each of the twenty-one patients in the MIT-BIH Arrhythmia Database, 100,000 time-step ECG data were divided into three as train, validation and test. For both datasets, 40,000 time-steps are separated for training data, 10,000 time-steps for validation data, and 50,000 time-steps as test data.

After the creation of the subsets, a benchmarking study including classical ML, classical RNN and DNN methods used in solving time series prediction problems was carried out in order to test the performance and comparison of the proposed hybrid method. In the following subsections, first detailed information is given how the chaotic time series data is obtained from the chaotic system equation sets. Secondly the details of the benchmarking setup, the evaluation metrics that will be used for comparison of the methods, and the test results obtained as a result of the experimental studies are given respectively.

3.1. Chaotic time series dataset

In this paper, two different chaotic time series dataset are used for the benchmark study on the prediction of chaotic time series. Firstly time series data were obtained by using frequently used classical dynamical systems: Lorenz, Rössler and Lorenz-like chaotic system equation sets. The differential equation set for the original Lorenz system are given in Equation (1) [53] and the well known chaotic parameters for this system have been taken as $\sigma = 10$, $p = 28$, and $\beta = 8/3$. By solving the differential equation set with initial conditions $(x_0, y_0, z_0) = (0.9, 0.9, 0)$, 100.000 total samples were obtained for each variable. The Lyapunov exponents of the system for these parameters and initial conditions are $LE_1 = 0.727225$, $LE_2 = 0.054129$, and $LE_3 = -14.448021$. At least one positive Lyapunov exponent indicates that the system is chaotic [54].

$$\dot{x} = \sigma(y - x) \tag{1a}$$

$$\dot{y} = rx - xz - y \tag{1b}$$

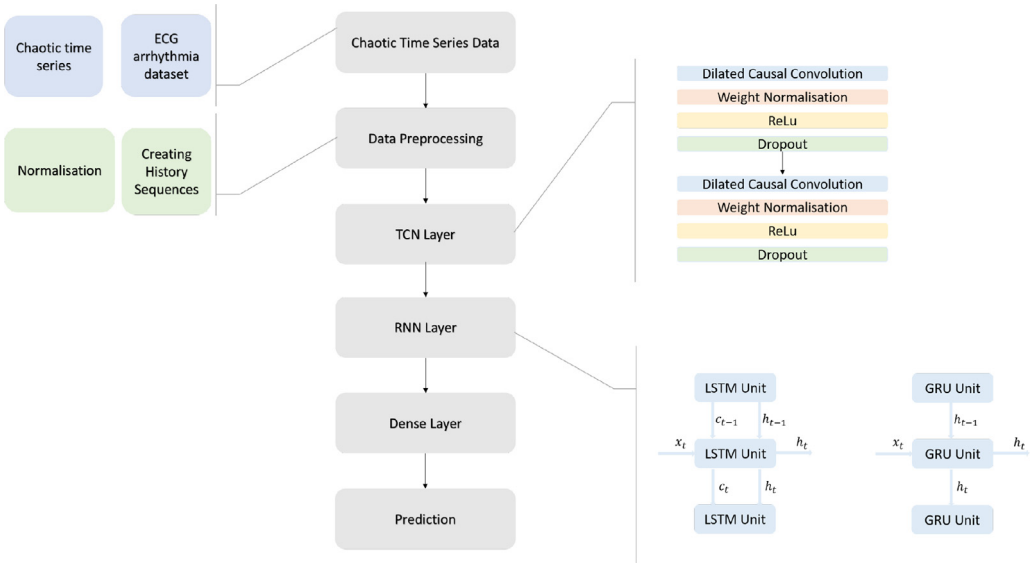


Fig. 4. Proposed hybrid approach of TCN-RNN.

$$\dot{z} = xy - \beta z \tag{1c}$$

For the Rössler chaotic equation set, with the parameters of $a = 0.2$, $b = 0.2$, and $c = 5.7$ was used for the Eq. (2) [55]. Initial conditions are chosen as $(x_0, y_0, z_0) = (0.1, 0.1, 0.1)$, and (2) is solved to obtain 100.000 samples for each variable. Calculated Lyapunov exponents as $LE_1 = 0.067146$, $LE_2 = 0.003444$, and $LE_3 = -5.401328$ prove that obtained time series are chaotic.

$$\dot{x} = -y - z \tag{2a}$$

$$\dot{y} = x + ay \tag{2b}$$

$$\dot{z} = b + z(x - c) \tag{2c}$$

In addition to these two chaotic system datasets, a Lorenz-like chaotic system, was also used for benchmarking [56]. The equation of this system, is given in (3). The parameters determined as $\alpha = 20$, $b = 4$ and $c = 32$ and the initial conditions are $(x_0, y_0, z_0) = (10, 10, 50)$. This chaotic equation set is also solved and obtained time series have 100.000 samples for each variable with the Lyapunov exponents of $LE_1 = 2.766436$, $LE_2 = 0.007600$, and $LE_3 = -18.774037$, indicating that the system is chaotic.

$$\dot{x} = \alpha(x - y) \tag{3a}$$

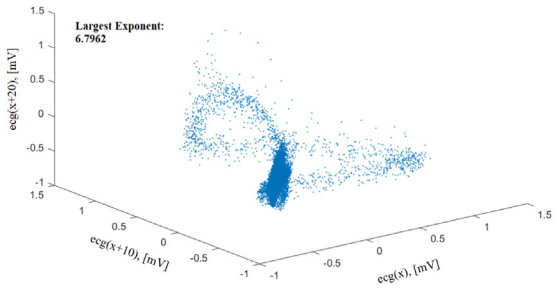
$$\dot{y} = (xz - cy) \tag{3b}$$

$$\dot{z} = x^2 - bz \tag{3c}$$

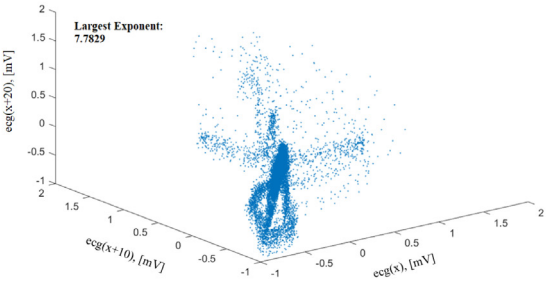
In the study, three chaotic equation sets were used for chaotic time series generation. A total of nine different time series were obtained to be used for benchmarking by using three variables (x, y and z) in each equation set. Additionally, simulations were carried out with an ECG arrhythmia dataset, which is not algorithmically generated, but real-life data. It has been accepted in the literature that arrhythmic ECG signals are chaotic data [57]. There are ECG data records of twenty-one people with arrhythmia in the dataset used [49,50]. This ECG dataset was prepared by digitizing the two-channel ambulatory ECG recordings of a 24-hour period with a sampling frequency of 360 samples per second. When the studies in the literature are examined, it is seen that all ECG data obtained from patients, including normal ECG data, have chaotic behaviors [57–61]. However, it has been observed as a result of experimental studies that the ECG data of people suffering from diseases such as arrhythmia and atrial fibrillation exhibit more chaotic behavior than the normal sinus ECG rhythm. For this reason, MIT-BIH Arrhythmia Database was used for real-time chaotic time series in this study. First of all, Lyapunov exponents of the ECG records were calculated to verify that the data used had chaotic properties [54]. The fact that the largest Lyapunov exponent obtained is positive indicates that this time series is chaotic. The exponents were calculated by tracking the distances between the orbits in 3D phase portrait with 10 samples of phase shift and the chaotic behavior of these ECG signals was verified. As an example, the phase portraits of 3 people and the largest lyapunov exponents are given in Fig. 5. In the study, the amplitude of the signal was estimated 2 ms ahead by using input data sequences with 28 ms history window. The data of Patient 111 and Patient 118 from 23 patients in the dataset were excluded from the simulations due to the fact that the models could not be trained correctly during learning and they produced outlier values during the benchmark study.

3.2. Benchmark setup

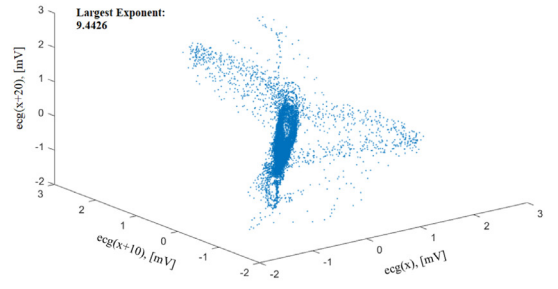
In order to examine the performance of the proposed hybrid architecture in the chaotic time series prediction, a comparison has been made with twelve different methods, which are used for time series prediction in the literature, including classical ML, DNN and RNN methods. In the classical ML and DNN methods used, the Grid Search algorithm [62] was used to determine hyper-parameters such as the number of layers and the activation function. Likewise, hyper-parameters such as the number of layers, unit size, filter size, and optimizer of proposed hybrid methods were also obtained by using the Grid Search algorithm. Grid Search was applied separately for nine different chaotic time series obtained from differential equations and chaotic time series which are obtained from twenty-one



(a) Patient 100, largest Lyapunov exponent is 6.7962.



(b) Patient 114, largest Lyapunov exponent is 7.7829.



(c) Patient 122, largest Lyapunov exponent is 9.4426.

Fig. 5. Phase-Shifted portraits of 3 patients and the largest lyapunov exponents confirming their chaotic natures.

patients in ECG arrhythmia dataset, and the models were trained using different hyper-parameters for these two types of datasets. The performances of different unit/filter size, activation function, loss function, optimizer and batch size combinations for 12 ML, DNN and RNN methods were compared with the Grid Search algorithm. For hyper-parameter selection grid search, values in the range of 32–512 for Unit/Filter Size and values in the range of 128–1024 for batch size were compared and the activation function was tested for ReLU, Tanh, Sigmoid, Leaky ReLU, Gaussian and Exponential Linear Unit functions. During the grid search evaluations, Mean Squared Error, Mean Absolute Error, Mean Absolute Percentage Error and Mean Squared Logarithmic Error were tested for the loss function. Finally, Adam, SGD, RMSprop, Adadelta and Nadam were compared for the optimizer selection. Grid search was performed for all 12 methods using 200 epochs. For all the methods used in the study, it was seen that the most successful loss function was mean squared error, and the most successful optimizer was Adam optimizer, therefore all DNN models were trained using these hyper-parameters. However, the successful activation function, unit/filter size and batch size hyper-parameters, were different for the selected methods and trainings were carried out with the most successful parameters. The model parameters that gives the best prediction performance and used in training of each ML and DNN model for both datasets can be seen in Tables 1 and 2. Training and test processes for benchmarking were carried out by Python programming language using Keras Application Interface. The device specifications used for the training are Intel(R) Core(TM) i7-10750H CPU, 32 GB random-access memory(RAM) and NVIDIA GeForce RTX2060 graphics processing unit (GPU). After the data normalization process, the trainings were carried out with a patience value of 50 for 200 epochs, with the network hyper-parameters given in Tables 1 and 2, using 10-time step history windows. By using the patience feature in the Python Keras library, it is aimed to terminate the training early in cases where it has not made any improvement for a certain number of epochs and thus to prevent the possibility of overfitting. At the end of the training processes, the models with the lowest validation loss were saved and used in the test processes.

3.3. Evaluation metrics

Evaluation metrics are standard metrics used in the literature to measure the accuracy of systems designed to solve a problem. In the literature, there are many numerical metrics used for time series prediction problems. The most commonly used metrics and the ones that will be used for benchmarking comparison purposes in this study are root mean square error (RMSE), mean absolute error (MAE), and coefficient of determination (R^2) [63–65]. The equations for these evaluation metrics, where n is the total number of the predicted samples, are given in Eqs. (4)–(6) respectively.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(predicted_i - measured_i)^2} \tag{4}$$

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|predicted_i - measured_i| \tag{5}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(predicted_i - measured_i)^2}{\sum_{i=1}^{n}(predicted_i - \frac{1}{n}\sum_{i=1}^{n}predicted_i)^2} \tag{6}$$

3.4. Simulation results

In this study, two different hybrid DNN architectures with similar structures are proposed for chaotic time series prediction and compared with 10 different learning methods in a benchmarking study. The first hybrid model proposed is the architecture built using an LSTM layer and a Dense layer at the output of the TCN layer, as seen in Fig. 4. The second is a model created using the GRU layer at the end of the TCN layer output. These two different TCN with RNN architectures (TCN-LSTM and TCN-GRU) were trained with the selected hyper-parameters using 10 previous time-steps as history for one time-step of prediction horizon. After the training process was completed, the models with the lowest validation loss were saved and the performance comparison of the methods was carried out with the test data. While performing the performance comparison, the RMSE, MAE, and R^2 score metrics were used.

Table 1
Hyper-parameters of the benchmarking methods for chaotic time series obtained from differential equations.

Methods	Layers	Unit/ Filter size	Activation function	Loss function	Optimizer	Batch size
Multi Layer Perceptron (MLP)	1	100	ReLU	MSE	Adam	256
Radial Based Function (RBF)	1	512	Gaussian	MSE	Adam	256
WaveNet	1	256	ReLU	MSE	Adam	512
TCN	1	32	ReLU	MSE	Adam	256
LSTM	1	64	Tanh	MSE	Adam	128
GRU	1	32	Tanh	MSE	Adam	128
TCN-TCN	2	64-128	ReLU-ReLU	MSE	Adam	128
LSTM-LSTM	2	256-32	Tanh-Tanh	MSE	Adam	128
GRU-GRU	2	256-128	Tanh-Tanh	MSE	Adam	256
LSTM-GRU	1-1	128-64	Tanh-Tanh	MSE	Adam	256
TCN-GRU	1-1	32-128	ReLU-Tanh	MSE	Adam	128
TCN-LSTM	1-1	64-64	ReLU-Tanh	MSE	Adam	128

Table 2
Hyper-parameters of the benchmarking methods for chaotic time series obtained from ECG arrhythmia dataset.

Methods	Layers	Unit/ Filter size	Activation function	Loss function	Optimizer	Batch size
Multi Layer Perceptron (MLP)	1	100	ReLU	MSE	Adam	256
Radial Based Function (RBF)	1	512	Gaussian	MSE	Adam	256
WaveNet	1	256	ReLU	MSE	Adam	512
TCN	1	256	ReLU	MSE	Adam	128
LSTM	1	256	Tanh	MSE	Adam	128
GRU	1	128	Tanh	MSE	Adam	128
TCN-TCN	2	128-64	ReLU-ReLU	MSE	Adam	512
LSTM-LSTM	2	256-32	Tanh-Tanh	MSE	Adam	256
GRU-GRU	2	256-128	Tanh-Tanh	MSE	Adam	256
LSTM-GRU	1-1	256-256	Tanh-Tanh	MSE	Adam	256
TCN-GRU	1-1	128-256	ReLU-Tanh	MSE	Adam	128
TCN-LSTM	1-1	64-64	ReLU-Tanh	MSE	Adam	256

The test results of the chaotic time series obtained from differential equations are given in Fig. 6, Fig. 7, and Fig. 8 respectively and in Tables 3–5 in detail. When the results of the benchmarking study are examined, it can been seen in Fig. 6 that the lowest average RMSE value for 9 different chaotic time series, was obtained with the proposed TCN-RNN models. At the same time, it was also observed that the lowest RMSE values for seven dataset obtained by the TCN-LSTM method. In terms of MAE values (in Fig. 7), obtaining both the lowest average MAE value and the lowest MAE value in 7 separate datasets proves that the proposed TCN-LSTM method gives more accurate prediction for chaotic time series. In addition to these, the proposed TCN-GRU model also obtained the second most successful results. These results show that TCN with RNN models have achieved high performance in predicting time series obtained from chaotic differential equation sets.

The test results of the chaotic time series obtained from ECG readings of 21 arrhythmia patients are given in Fig. 9, Fig. 10, and Fig. 11 respectively and in Tables 6–8 in detail. In the benchmark study, 12 different methods were compared, and TCN-LSTM and TCN-GRU methods, in which TCN and RNN structures were used together, achieved the lowest RMSE value (in Fig. 9) in 20 of 21 patients. In addition, the lowest average RMSE value of 0.0082 was obtained using TCN-LSTM, while the second lowest average RMSE value was obtained with the TCN-GRU method with 0.0084. These results show that the use of TCN and RNN structures together is more successful than classical ML, RNN and DNN methods for chaotic time series prediction. When the MAE values are analyzed in Table 7, it is clearly seen that the lowest average MAE values were obtained with proposed TCN-GRU and TCN-LSTM models.

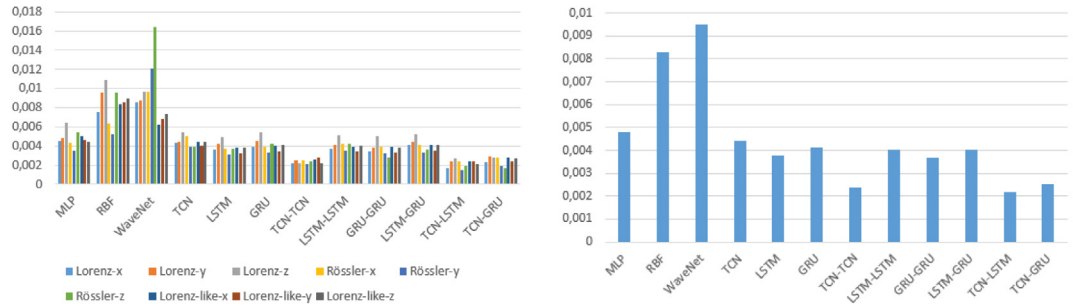
When the average RMSE value was calculated for 30 different time series using the RMSE values in all tables, it is seen that TCN-LSTM and TCN-GRU were the two models with the highest prediction performance. These results show that the proposed TCN-RNN models give more accurate performance for the chaotic time series prediction compared to both classical ML and DNN models in the literature.

After analyzing the RMSE values, the graphs in which the actual and predicted values with classical DNN methods and the proposed TCN with RNN method are plotted for the Rössler-y chaotic time series, where the best prediction performance is obtained with proposed TCN-LSTM method are given in Figs. 12 and 13. Similar graphs are also given for the Patient 103 chaotic time series with classical DNN methods and the proposed TCN with RNN method, where the best prediction performance is obtained for TCN-LSTM, in Figs. 14 and 15, respectively.

As a result of the Friedman test [66,67] performed using the RMSE results, which was primarily examined in the study, it can be seen that one of the methods has a statistically significant success difference for both the chaotic time-series dataset and the ECG arrhythmia dataset. The Friedman test results are also given in detail in Table 11. When the methods with the significant difference compared to the classical ML and DNN methods is found using the post-hoc Nemeyni test [68], it is seen that these methods include the recommended TCN with RNN methods for both data types. The detailed result table of Nemeyni tests is given in Tables 9 and 10.

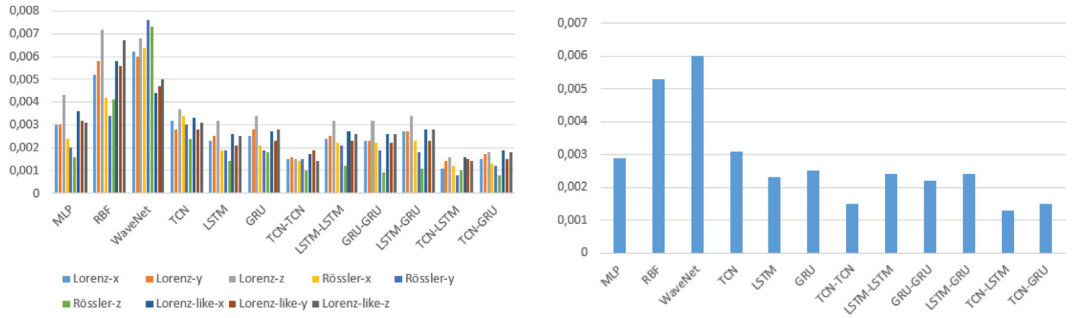
4. Conclusion and future work

In this study, a hybrid method, consisting of TCN and RNN architectures is proposed for the chaotic time series prediction. In this way, features belonging to both short-term changes and long-term dependencies along with non-linear features can be extracted with higher success and lower computational load compared to other stack architectures. In order to test the performance of the proposed model, nine different data sets, which were obtained by using three different chaotic differential equation sets and ECG time series obtained from twenty-one patients were used. 50% of each of these 30 different time series were used



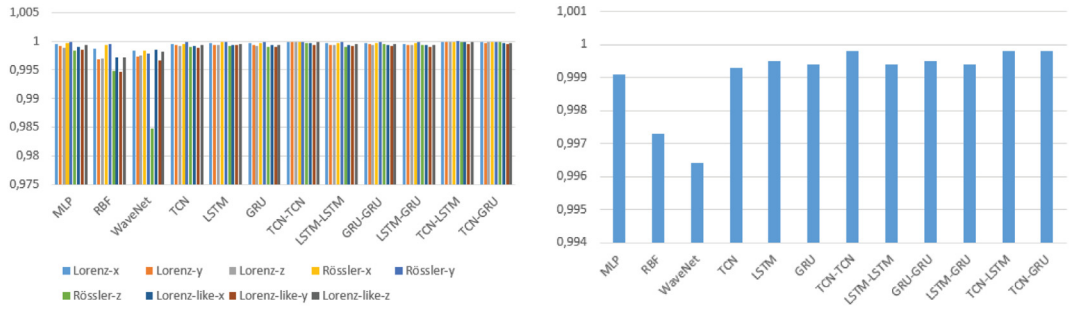
(a) RMSE values for 9 different chaotic time series. (b) Average RMSE values.

Fig. 6. RMSE values obtained from the benchmarking study using chaotic time series.



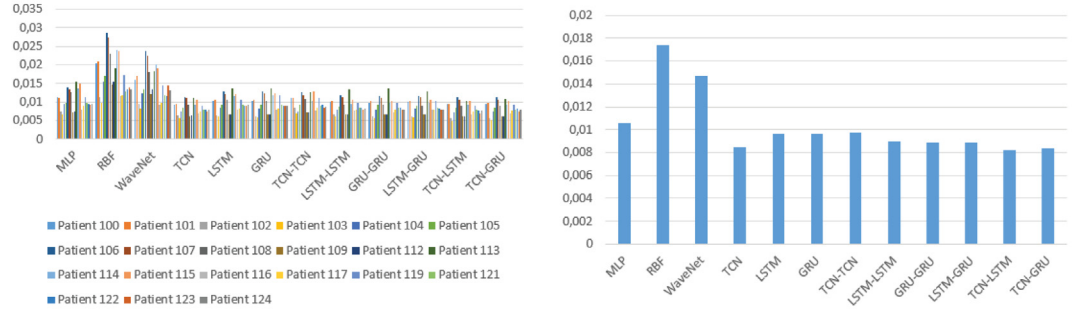
(a) MAE values for 9 different chaotic time series. (b) Average MAE values.

Fig. 7. MAE values obtained from the benchmarking study using chaotic time series.



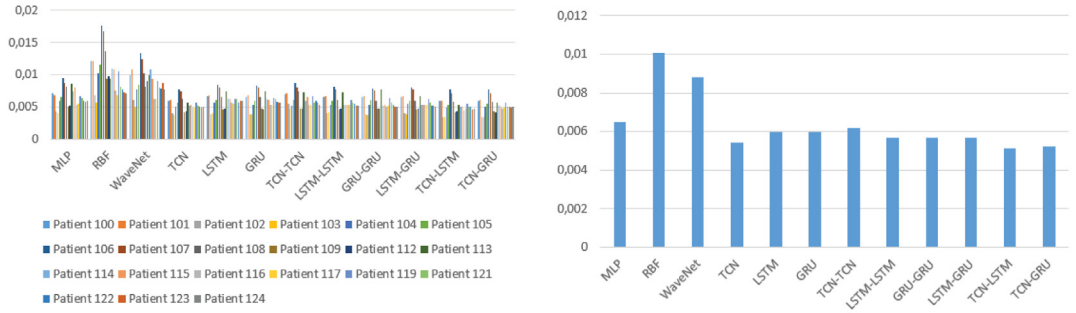
(a) R^2 values for 9 different chaotic time series. (b) Average R^2 values.

Fig. 8. R^2-score values obtained from the benchmarking study using chaotic time series.



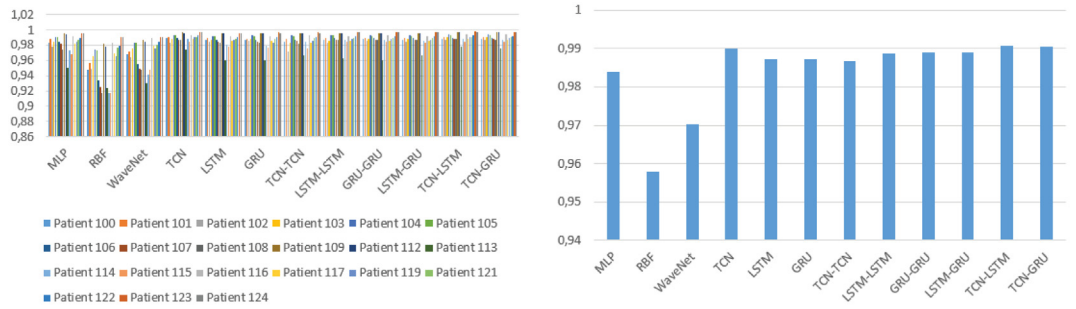
(a) RMSE values for using 21 different patients data. (b) Average RMSE values.

Fig. 9. RMSE values from the benchmarking study chaotic time series obtained from ECG arrhythmia dataset.



(a) MAE values for using 21 different patients data. (b) Average MAE values.

Fig. 10. MAE values from the benchmarking study chaotic time series obtained from ECG arrhythmia dataset.



(a) R^2 values for using 21 different patients data. (b) Average R^2 values.

Fig. 11. R^2-score values from the benchmarking study chaotic time series obtained from ECG arrhythmia dataset.

Table 3
RMSE values obtained from the benchmarking study using 9 different chaotic time series.

Dataset	MLP	RBF	WaveNet	TCN	LSTM	GRU	TCN-TCN	LSTM-LSTM	GRU-GRU	LSTM-GRU	TCN-LSTM	TCN-GRU
Lorenz-x	0.0045	0.0075	0.0086	0.0043	0.0036	0.0039	0.0022	0.0037	0.0034	0.0041	**0.0017**	0.0023
Lorenz-y	0.0048	0.0096	0.0088	0.0044	0.0042	0.0045	0.0025	0.0041	0.0038	0.0044	**0.0024**	0.0029
Lorenz-z	0.0064	0.0109	0.0097	0.0054	0.0049	0.0054	**0.0022**	0.0051	0.0050	0.0052	0.0027	0.0028
Rössler-x	0.0043	0.0063	0.0097	0.0050	0.0037	0.0039	0.0025	0.0042	0.0039	0.0041	**0.0024**	0.0028
Rössler-y	0.0035	0.0052	0.0121	0.0039	0.0031	0.0033	0.0021	0.0035	0.0032	0.0033	**0.0015**	0.0019
Rössler-z	0.0054	0.0096	0.0164	0.0039	0.0037	0.0042	0.0024	0.0042	0.0028	0.0036	0.0019	**0.0017**
Lorenz-like-x	0.0050	0.0084	0.0062	0.0044	0.0038	0.0040	0.0026	0.0039	0.0039	0.0041	**0.0024**	0.0028
Lorenz-like-y	0.0046	0.0086	0.0068	0.0040	0.0032	0.0034	0.0028	0.0034	0.0033	0.0035	**0.0024**	**0.0024**
Lorenz-like-z	0.0044	0.0090	0.0073	0.0044	0.0038	0.0041	0.0022	0.0040	0.0038	0.0041	**0.0021**	0.0027
Average	0.0048	0.0083	0.0095	0.0044	0.0038	0.0041	0.0024	0.0040	0.0037	0.0040	**0.0022**	0.0025

Table 4
MAE values obtained from the benchmarking study using 9 different chaotic time series.

Dataset	MLP	RBF	WaveNet	TCN	LSTM	GRU	TCN-TCN	LSTM-LSTM	GRU-GRU	LSTM-GRU	TCN-LSTM	TCN-GRU
Lorenz-x	0.0030	0.0052	0.0062	0.0032	0.0023	0.0025	0.0015	0.0024	0.0023	0.0027	**0.0011**	0.0015
Lorenz-y	0.0030	0.0058	0.0060	0.0028	0.0025	0.0028	0.0016	0.0025	0.0023	0.0027	**0.0014**	0.0017
Lorenz-z	0.0043	0.0072	0.0068	0.0037	0.0032	0.0034	**0.0015**	0.0032	0.0032	0.0034	0.0016	0.0018
Rössler-x	0.0024	0.0042	0.0064	0.0034	0.0019	0.0021	0.0014	0.0022	0.0022	0.0023	**0.0012**	0.0013
Rössler-y	0.0020	0.0034	0.0076	0.0030	0.0019	0.0019	0.0015	0.0021	0.0019	0.0018	**0.0008**	0.0012
Rössler-z	0.0016	0.0041	0.0073	0.0024	0.0014	0.0018	0.0010	0.0012	0.0009	0.0011	0.0010	**0.0008**
Lorenz-like-x	0.0036	0.0058	0.0044	0.0033	0.0026	0.0027	0.0017	0.0027	0.0026	0.0028	**0.0016**	0.0019
Lorenz-like-y	0.0032	0.0056	0.0047	0.0028	0.0021	0.0023	0.0019	0.0023	0.0022	0.0023	**0.0015**	**0.0015**
Lorenz-like-z	0.0031	0.0067	0.0050	0.0031	0.0025	0.0028	**0.0014**	0.0026	0.0026	0.0028	**0.0014**	0.0018
Average	0.0029	0.0053	0.0060	0.0031	0.0023	0.0025	0.0015	0.0024	0.0022	0.0024	**0.0013**	0.0015

for model training and 50% for testing the trained models. Simulation studies were carried out in which a total of twelve methods consisting of classical ML, classical DNN and RNN were used to compare with the proposed TCN with RNN method. In order to obtain the best results from the benchmarking of the methods, hyper-parameters selections were made. Grid search algorithm was used in this study for hyper-parameters selections and the parameters with the lowest loss value were used in the training of the models. The results were analyzed using RMSE, MAE and R^2 score evaluation metrics. When the evaluation metrics obtained were examined, it was seen that TCN-LSTM, one of the suggested methods, with the lowest RMSE value of 0.0015 and the highest R^2 score, predicted Rössler-y data more accurately than other methods used in comparison. In addition, when the average of the

Table 5
R^2 score values obtained from the benchmarking study using 9 different chaotic time series.

Dataset	MLP	RBF	WaveNet	TCN	LSTM	GRU	TCN-TCN	LSTM-LSTM	GRU-GRU	LSTM-GRU	TCN-LSTM	TCN-GRU
Lorenz-x	0.9995	0.9987	0.9983	0.9996	0.9997	0.9997	**0.9999**	0.9997	0.9997	0.9996	0.9999	**0.9999**
Lorenz-y	0.9992	0.9968	0.9973	0.9993	0.9994	0.9993	**0.9998**	0.9994	0.9995	0.9993	**0.9998**	0.9997
Lorenz-z	0.9989	0.9970	0.9976	0.9992	0.9994	0.9992	**0.9999**	0.9993	0.9994	0.9993	0.9998	0.9998
Rössler-x	0.9997	0.9993	0.9983	0.9995	0.9998	0.9997	**0.9999**	0.9997	0.9997	0.9997	**0.9999**	**0.9999**
Rössler-y	0.9998	0.9996	0.9979	0.9998	0.9999	0.9998	0.9999	0.9998	0.9999	0.9998	**1.0000**	0.9999
Rössler-z	0.9983	0.9948	0.9847	0.9991	0.9992	0.9990	0.9997	0.9990	0.9996	0.9993	**0.9998**	**0.9998**
Lorenz-like-x	0.9990	0.9972	0.9985	0.9992	0.9994	0.9994	0.9997	0.9994	0.9994	0.9994	**0.9998**	0.9997
Lorenz-like-y	0.9985	0.9947	0.9966	0.9989	0.9993	0.9991	0.9994	0.9992	0.9992	0.9991	**0.9996**	**0.9996**
Lorenz-like-z	0.9993	0.9972	0.9982	0.9993	0.9995	0.9994	**0.9998**	0.9995	0.9995	0.9994	**0.9998**	0.9997
Average	0.9991	0.9973	0.9964	0.9993	0.9995	0.9994	**0.9998**	0.9994	0.9995	0.9994	**0.9998**	**0.9998**

Table 6
RMSE values from the benchmarking study using 21 different chaotic time series obtained from ECG arrhythmia dataset.

Dataset	MLP	RBF	WaveNet	TCN	LSTM	GRU	TCN-TCN	LSTM-LSTM	GRU-GRU	LSTM-GRU	TCN-LSTM	TCN-GRU
Patient 100	0.0114	0.0204	0.0160	**0.0093**	0.0104	0.0103	0.0110	0.0101	0.0099	0.0100	0.0095	0.0095
Patient 101	0.0110	0.0210	0.0170	**0.0096**	0.0105	0.0107	0.0110	0.0103	0.0102	0.0103	**0.0096**	0.0097
Patient 102	0.0074	0.0114	0.0096	0.0065	0.0064	0.0062	0.0086	0.0066	0.0061	0.0062	**0.0057**	**0.0057**
Patient 103	0.0067	0.0098	0.0082	0.0056	0.0061	0.0059	0.0069	0.0063	0.0057	0.0058	**0.0050**	0.0051
Patient 104	0.0095	0.0154	0.0123	0.0076	0.0086	0.0082	0.0076	0.0080	0.0079	0.0082	**0.0073**	0.0074
Patient 105	0.0099	0.0170	0.0135	0.0086	0.0093	0.0092	0.0094	0.0088	0.0092	0.0091	**0.0084**	0.0086
Patient 106	0.0140	0.0287	0.0237	0.0114	0.0129	0.0129	0.0127	0.0118	0.0117	0.0117	**0.0113**	0.0114
Patient 107	0.0134	0.0273	0.0226	0.0111	0.0122	0.0125	0.0119	0.0113	0.0112	0.0113	**0.0106**	0.0107
Patient 108	0.0127	0.0230	0.0182	0.0092	0.0105	0.0104	0.0109	0.0092	0.0093	0.0091	**0.0089**	0.0090
Patient 109	0.0073	0.0148	0.0122	0.0063	0.0067	0.0067	0.0073	0.0067	0.0067	0.0067	**0.0062**	**0.0062**
Patient 112	0.0076	0.0155	0.0133	0.0064	0.0067	0.0067	0.0071	0.0068	0.0068	0.0068	**0.0063**	**0.0063**
Patient 113	0.0154	0.0192	0.0184	0.0110	0.0137	0.0138	0.0127	0.0134	0.0138	0.0128	**0.0102**	0.0109
Patient 114	0.0138	0.0239	0.0201	**0.0092**	0.0117	0.0119	0.0104	0.0096	0.0097	0.0098	**0.0092**	0.0094
Patient 115	0.0150	0.0238	0.0190	0.0105	0.0122	0.0125	0.0128	0.0105	0.0103	0.0107	**0.0102**	0.0103
Patient 116	0.0080	0.0115	0.0094	0.0070	0.0081	0.0081	0.0077	0.0077	0.0073	0.0079	**0.0067**	0.0069
Patient 117	0.0089	0.0119	0.0098	**0.0075**	0.0082	0.0082	0.0086	0.0081	0.0080	0.0081	**0.0075**	0.0078
Patient 119	0.0113	0.0172	0.0144	**0.0091**	0.0105	0.0119	0.0110	0.0099	0.0099	0.0104	**0.0091**	0.0094
Patient 121	0.0098	0.0130	0.0118	0.0080	0.0094	0.0093	0.0089	0.0086	0.0086	0.0086	**0.0079**	0.0080
Patient 122	0.0095	0.0133	0.0117	0.0079	0.0089	0.0091	0.0092	0.0085	0.0084	0.0083	**0.0078**	0.0082
Patient 123	0.0094	0.0139	0.0144	0.0074	0.0090	0.0089	0.0086	0.0080	0.0081	0.0079	**0.0069**	0.0075
Patient 124	0.0095	0.0134	0.0132	0.0081	0.0093	0.0089	0.0087	0.0082	0.0079	0.0081	**0.0077**	0.0081
Average	0.0106	0.0174	0.0147	0.0085	0.0096	0.0096	0.0097	0.0090	0.0089	0.0089	**0.0082**	0.0084

Table 7
MAE values from the benchmarking study using 21 different chaotic time series obtained from ECG arrhythmia dataset.

Dataset	MLP	RBF	WaveNet	TCN	LSTM	GRU	TCN-TCN	LSTM-LSTM	GRU-GRU	LSTM-GRU	TCN-LSTM	TCN-GRU
Patient 100	0.0071	0.0121	0.0099	**0.0060**	0.0067	0.0066	0.0070	0.0066	0.0065	0.0066	**0.0060**	**0.0060**
Patient 101	0.0069	0.0122	0.0109	0.0061	0.0069	0.0069	0.0072	0.0067	0.0067	0.0067	**0.0060**	0.0061
Patient 102	0.0044	0.0069	0.0061	0.0041	0.0039	0.0039	0.0055	0.0041	0.0039	0.0040	**0.0035**	**0.0035**
Patient 103	0.0041	0.0056	0.0051	0.0037	0.0040	0.0039	0.0048	0.0042	0.0037	0.0039	**0.0034**	**0.0034**
Patient 104	0.0060	0.0103	0.0078	0.0051	0.0057	0.0054	0.0052	0.0054	0.0054	0.0055	**0.0049**	0.0050
Patient 105	0.0065	0.0115	0.0085	0.0057	0.0061	0.0060	0.0061	0.0059	0.0061	0.0060	**0.0054**	0.0055
Patient 106	0.0095	0.0177	0.0134	0.0078	0.0084	0.0083	0.0087	0.0081	0.0079	0.0080	**0.0077**	0.0078
Patient 107	0.0087	0.0167	0.0125	0.0074	0.0080	0.0080	0.0080	0.0077	0.0076	0.0077	**0.0071**	0.0072
Patient 108	0.0082	0.0137	0.0102	0.0062	0.0066	0.0065	0.0075	0.0061	0.0060	0.0060	**0.0058**	**0.0058**
Patient 109	0.0050	0.0094	0.0082	**0.0042**	0.0046	0.0047	0.0048	0.0046	0.0047	0.0046	**0.0042**	0.0043
Patient 112	0.0052	0.0098	0.0090	0.0044	0.0047	0.0046	0.0047	0.0048	0.0047	0.0048	0.0043	**0.0042**
Patient 113	0.0086	0.0094	0.0099	0.0057	0.0075	0.0074	0.0073	0.0073	0.0077	0.0067	**0.0053**	0.0056
Patient 114	0.0074	0.0110	0.0109	0.0052	0.0062	0.0063	0.0059	0.0054	0.0052	0.0054	**0.0050**	0.0052
Patient 115	0.0080	0.0109	0.0094	0.0053	0.0062	0.0061	0.0066	0.0053	0.0053	0.0054	**0.0050**	0.0051
Patient 116	0.0054	0.0076	0.0062	0.0049	0.0056	0.0054	0.0053	0.0053	0.0051	0.0053	**0.0045**	0.0047
Patient 117	0.0055	0.0068	0.0062	0.0049	0.0055	0.0054	0.0053	0.0053	0.0052	0.0054	**0.0048**	0.0049
Patient 119	0.0067	0.0105	0.0090	0.0056	0.0063	0.0064	0.0067	0.0061	0.0064	0.0062	**0.0055**	0.0057
Patient 121	0.0064	0.0082	0.0080	0.0052	0.0062	0.0062	0.0057	0.0057	0.0056	0.0056	**0.0051**	**0.0051**
Patient 122	0.0060	0.0077	0.0079	0.0051	0.0057	0.0058	0.0060	0.0055	0.0054	0.0052	**0.0050**	**0.0050**
Patient 123	0.0058	0.0073	0.0087	0.0049	0.0059	0.0057	0.0056	0.0052	0.0051	0.0052	**0.0046**	0.0049
Patient 124	0.0059	0.0072	0.0078	0.0051	0.0059	0.0057	0.0054	0.0052	0.0051	0.0050	**0.0048**	0.0050
Average	0.0065	0.0101	0.0088	0.0054	0.0060	0.0060	0.0062	0.0057	0.0057	0.0057	**0.0051**	0.0052

Table 8
R^2 score values from the benchmarking study using 21 different chaotic time series obtained from ECG arrhythmia dataset.

Dataset	MLP	RBF	WaveNet	TCN	LSTM	GRU	TCN-TCN	LSTM-LSTM	GRU-GRU	LSTM-GRU	TCN-LSTM	TCN-GRU
Patient 100	0.9836	0.9478	0.9677	0.9891	0.9863	0.9865	0.9847	0.9871	0.9876	0.9874	**0.9886**	**0.9886**
Patient 101	0.9880	0.9564	0.9716	0.9910	0.9891	0.9888	0.9880	0.9896	0.9898	0.9896	**0.9910**	0.9907
Patient 102	0.9784	0.9491	0.9637	0.9835	0.9839	0.9851	0.9712	0.9829	0.9853	0.9847	**0.9873**	0.9872
Patient 103	0.9840	0.9659	0.9760	0.9888	0.9867	0.9878	0.9833	0.9861	0.9885	0.9879	**0.9911**	0.9907
Patient 104	0.9904	0.9743	0.9836	0.9937	0.9921	0.9928	0.9937	0.9930	0.9933	0.9928	**0.9943**	0.9942
Patient 105	0.9910	0.9734	0.9832	0.9931	0.9921	0.9921	0.9918	0.9929	0.9921	0.9923	**0.9935**	0.9932
Patient 106	0.9844	0.9344	0.9553	0.9897	0.9868	0.9868	0.9872	0.9890	0.9891	0.9891	**0.9897**	**0.9897**
Patient 107	0.9819	0.9251	0.9486	0.9875	0.9850	0.9843	0.9858	0.9871	0.9874	0.9873	**0.9887**	0.9884
Patient 108	0.9748	0.9174	0.9480	0.9868	0.9827	0.9831	0.9813	0.9867	0.9864	0.9869	**0.9877**	0.9872
Patient 109	0.9956	0.9817	0.9875	0.9967	0.9962	0.9962	0.9956	0.9963	0.9962	0.9962	**0.9968**	**0.9968**
Patient 112	0.9947	0.9780	0.9838	0.9962	0.9959	0.9959	0.9953	0.9958	0.9958	0.9958	**0.9964**	**0.9964**
Patient 113	0.9509	0.9241	0.9300	0.9748	0.9610	0.9607	0.9669	0.9629	0.9606	0.9663	**0.9786**	0.9754
Patient 114	0.9726	0.9176	0.9415	0.9877	0.9803	0.9797	0.9844	0.9867	0.9864	0.9861	**0.9878**	0.9872
Patient 115	0.9674	0.9176	0.9475	0.9840	0.9785	0.9774	0.9762	0.9841	0.9845	0.9834	**0.9850**	0.9847
Patient 116	0.9920	0.9834	0.9889	0.9938	0.9918	0.9918	0.9926	0.9925	0.9933	0.9922	**0.9944**	0.9941
Patient 117	0.9825	0.9688	0.9789	0.9876	0.9852	0.9851	0.9836	0.9857	0.9861	0.9856	**0.9876**	0.9865
Patient 119	0.9850	0.9656	0.9758	**0.9904**	0.9873	0.9835	0.9859	0.9886	0.9886	0.9873	0.9903	0.9897
Patient 121	0.9866	0.9766	0.9806	0.9912	0.9877	0.9879	0.9890	0.9896	0.9898	0.9898	**0.9913**	0.9911
Patient 122	0.9896	0.9796	0.9843	0.9927	0.9908	0.9905	0.9903	0.9917	0.9919	0.9921	**0.9929**	0.9922
Patient 123	0.9960	0.9912	0.9905	0.9975	0.9963	0.9964	0.9966	0.9971	0.9970	0.9972	**0.9978**	0.9974
Patient 124	0.9952	0.9904	0.9908	0.9965	0.9954	0.9958	0.9960	0.9964	0.9967	0.9965	**0.9968**	0.9965
Average	0.9840	0.9580	0.9704	0.9901	0.9872	0.9871	0.9866	0.9887	0.9889	0.9889	**0.9908**	0.9904

Table 9
Post-hoc Nemeyni test results for chaotic time series.

	MLP	RBF	WaveNet	TCN	LSTM	GRU	TCN-TCN	LSTM-LSTM	GRU-GRU	LSTM-GRU	TCN-LSTM	TCN-GRU
MLP	1.000	0.900	0.900	0.900	0.107	0.900	0.001	0.775	0.089	0.900	0.001	0.001
RBF	0.900	1.000	0.900	0.900	0.003	0.351	0.001	0.149	0.002	0.271	0.001	0.001
WaveNet	0.900	0.900	1.000	0.900	0.004	0.396	0.001	0.175	0.003	0.309	0.001	0.001
TCN	0.900	0.900	0.900	1.000	0.351	0.900	0.005	0.900	0.309	0.900	0.001	0.011
LSTM	0.107	0.003	0.004	0.351	1.000	0.900	0.900	0.900	0.900	0.900	0.651	0.900
GRU	0.900	0.351	0.396	0.900	0.900	1.000	0.107	0.900	0.898	0.900	0.017	0.175
TCN-TCN	0.001	0.001	0.001	0.005	0.900	0.107	1.000	0.271	0.900	0.149	0.900	0.900
LSTM-LSTM	0.775	0.149	0.175	0.900	0.900	0.900	0.271	1.000	0.900	0.900	0.061	0.396
GRU-GRU	0.089	0.002	0.003	0.309	0.900	0.898	0.900	0.900	1.000	0.900	0.693	0.900
LSTM-GRU	0.001	0.001	0.001	0.001	0.651	0.017	0.900	0.061	0.693	0.027	1.000	0.900
TCN-LSTM	0.001	0.001	0.001	0.011	0.900	0.175	0.900	0.396	0.900	0.236	0.900	1.000
TCN-GRU	0.900	0.271	0.309	0.900	0.900	0.900	0.149	0.900	0.900	1.000	0.027	0.236

Table 10
Post-hoc Nemeyni test results for ECG arryhytmia dataset.

	MLP	RBF	WaveNet	TCN	LSTM	GRU	TCN-TCN	LSTM-LSTM	GRU-GRU	LSTM-GRU	TCN-LSTM	TCN-GRU
MLP	1.000	0.614	0.900	0.001	0.589	0.743	0.900	0.002	0.001	0.001	0.001	0.001
RBF	0.614	1.000	0.900	0.001	0.002	0.004	0.022	0.001	0.001	0.001	0.001	0.001
WaveNet	0.900	0.900	1.000	0.001	0.064	0.125	0.344	0.001	0.001	0.001	0.001	0.001
TCN	0.001	0.001	0.001	1.000	0.006	0.002	0.001	0.743	0.900	0.871	0.900	0.900
LSTM	0.589	0.002	0.064	0.006	1.000	0.900	0.900	0.666	0.227	0.537	0.001	0.056
GRU	0.743	0.004	0.125	0.002	0.900	1.000	0.900	0.512	0.125	0.372	0.001	0.026
TCN-TCN	0.900	0.022	0.344	0.001	0.900	0.900	1.000	0.227	0.034	0.139	0.001	0.005
LSTM-LSTM	0.002	0.001	0.001	0.743	0.666	0.512	0.227	1.000	0.900	0.900	0.101	0.900
GRU-GRU	0.001	0.001	0.001	0.900	0.227	0.125	0.034	0.900	1.000	0.900	0.458	0.900
LSTM-GRU	0.001	0.001	0.001	0.900	0.001	0.001	0.001	0.101	0.458	0.171	1.000	0.794
TCN-LSTM	0.001	0.001	0.001	0.900	0.056	0.026	0.005	0.900	0.900	0.900	0.794	1.000
TCN-GRU	0.001	0.001	0.001	0.871	0.537	0.372	0.139	0.900	0.900	1.000	0.171	0.900

Table 11
Friedman test results obtained from RMSE metrics of the benchmarking study using different data.

Dataset	Test statistic	P-value
Chaotic time-series data	94.45	2.24×10^{-15}
ECG arrhythmia dataset	187.68	2.66×10^{-34}

RMSE value is calculated from 30 different chaotic time series, it is clearly seen that the TCN with RNN architectures have the lowest error rate. When the predicted and actual data plots are examined in addition to these, it is clearly seen that the prediction results obtained with the proposed TCN with RNN methods can closely follow the actual data for both Rössler-y data and ECG arrhythmia data.

Although there are similar studies [48] using hybrid methods in the literature, the proposed TCN with RNN hybrid method is a new perspective for the prediction of chaotic data. In future, various new DNN modules will be added to the proposed architecture and their contributions to the solution of this problem will be examined. Furthermore, whether the proposed architecture would contribute to predict different time series will be investigated by the researchers as future work. Also, the optimization of hyper-parameters with heuristic algorithms can be added to strengthen the robustness of the model. Validation of current and strengthened models can be increased by testing on different

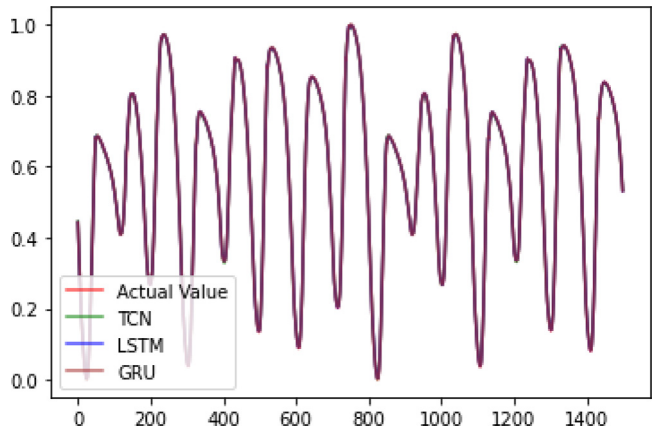


Fig. 12. Plotting normalized actual and predicted values with classical DNN methods for Rössler-*y* chaotic time series. (a) Actual values, (b) predicted values with TCN method, (c) predicted values with LSTM method, (d) predicted values with GRU method.

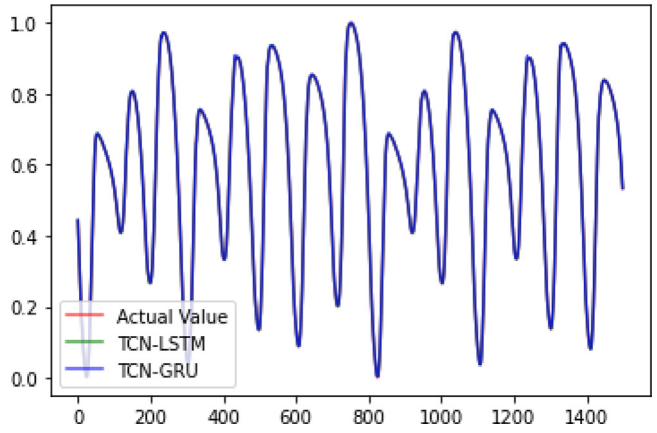


Fig. 13. Plotting normalized actual and predicted values with proposed TCN-RNN methods for Rössler-*y* chaotic time series.

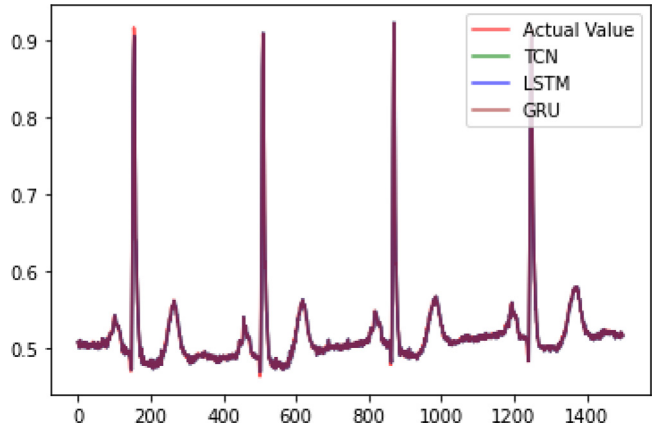


Fig. 14. Plotting normalized actual and predicted values with classical DNN methods for Patient 103. (a) Actual values, (b) predicted values with TCN method, (c) predicted values with LSTM method, (d) predicted values with GRU method.

real-life chaotic data (eg renewable energy generation, electricity load, stock market, solar activity).

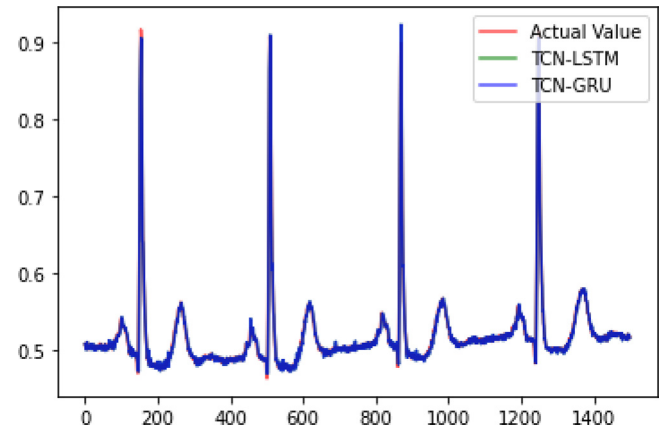


Fig. 15. Plotting normalized actual and predicted values with proposed TCN-RNN methods for Patient 103.

CRediT authorship contribution statement

Hatice Vildan Dudukcu: Conceptualization, Methodology, Software, Writing – original draft. **Murat Taskiran:** Conceptualization, Methodology, Software, Writing – original draft. **Zehra Gulru Cam Taskiran:** Conceptualization, Methodology, Writing – review & editing, Supervision. **Tulay Yildirim:** Conceptualization, Methodology, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Hatice Vildan Dudukcu reports equipment, drugs, or supplies was provided by Yildit Technical University Scientific Research Projects Coordinaton Unit.

Data availability

The authors do not have permission to share data.

Acknowledgments

This work has been supported by Yildiz Technical University Scientific Research Projects Coordination Unit, Turkey under project number FAP-2021-4221.

References

[1] M.Y. Ye, X.D. Wang, H.R. Zhang, Chaotic time series forecasting using online least squares support vector machine regression, Acta Phys. Sin. 54 (6) (2005) 2568–2573.
[2] Hongbo Meng, Changming Wang, Aijun Zhang, Jiandong Bao, A novel chaotic time series prediction method and its application to carrier vibration interference attitude prediction of stabilized platform, J. Vibroeng. 18 (1) (2016) 562–576.
[3] Nor Zila Abd Hamid, Mohd Salmi Md Noorani, New improved chaotic approach model application on forecasting ozone concentration time series, Sains Malays. 46 (8) (2017) 1333–1339.
[4] Said Jadid Abdulkadir, Suet-Peng Yong, Scaled UKF–NARX hybrid model for multi-step-ahead forecasting of chaotic time series data, Soft Comput. 19 (12) (2015) 3479–3496.
[5] Hector Rodriguez, Juan J. Flores, Luis A. Morales, Carlos Lara, Armando Guerra, Giovanni Manjarrez, Forecasting from incomplete and chaotic wind speed data, Soft Comput. 23 (20) (2019) 10119–10127.
[6] G. Peter Zhang, Time series forecasting using a hybrid ARIMA and neural network model, Neurocomputing 50 (2003) 159–175.
[7] U.V.B.R. Thissen, R. Van Brakel, A.P. De Weijer, W.J. Melssen, L.M.C. Buydens, Using support vector machines for time series prediction, Chemometr. Intell. Lab. Syst. 69 (1–2) (2003) 35–49.
[8] G. Peter Zhang, Douglas M. Kline, Quarterly time-series forecasting with neural networks, IEEE Trans. Neural Netw. 18 (6) (2007) 1800–1814.

[9] Weizhong Yan, Toward automatic time-series forecasting using neural networks, IEEE Trans. Neural Netw. Learn. Syst. 23 (7) (2012) 1028–1039.
[10] Nikolaos Kourentzes, Devon K. Barrow, Sven F. Crone, Neural network ensemble operators for time series forecasting, Expert Syst. Appl. 41 (9) (2014) 4235–4244.
[11] Chuanjin Jiang, Fugen Song, Forecasting chaotic time series of exchange rate based on nonlinear autoregressive model, in: 2010 2nd International Conference on Advanced Computer Control, vol. 5, IEEE, 2010, pp. 238–241.
[12] Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kanniainen, Moncef Gabbouj, Alexandros Iosifidis, Forecasting stock prices from the limit order book using convolutional neural networks, in: 2017 IEEE 19th Conference on Business Informatics, CBI, vol. 1, IEEE, 2017, pp. 7–12.
[13] Irena Koprinska, Dengsong Wu, Zheng Wang, Convolutional neural networks for energy time series forecasting, in: 2018 International Joint Conference on Neural Networks, IJCNN, IEEE, 2018, pp. 1–8.
[14] Ping-Huan Kuo, Chiou-Jye Huang, A high precision artificial neural networks model for short-term energy load forecasting, Energies 11 (1) (2018) 213.
[15] Hui Liu, Xiwei Mi, Yanfei Li, Smart deep learning based wind speed prediction model using wavelet packet decomposition, convolutional neural network and convolutional long short term memory network, Energy Convers. Manage. 166 (2018) 120–131.
[16] Chujie Tian, Jian Ma, Chunhong Zhang, Panpan Zhan, A deep neural network model for short-term load forecast based on long short-term memory network and convolutional neural network, Energies 11 (12) (2018) 3493.
[17] Mengmeng Cai, Manisa Pipattanasomporn, Saifur Rahman, Day-ahead building-level load forecasts using deep learning vs. traditional time-series techniques, Appl. Energy 236 (2019) 1078–1088.
[18] Zhipeng Shen, Yuanming Zhang, Jiawei Lu, Jun Xu, Gang Xiao, A novel time series forecasting model with deep learning, Neurocomputing 396 (2020) 302–313.
[19] Rohitash Chandra, Mengjie Zhang, Cooperative coevolution of elman recurrent neural networks for chaotic time series prediction, Neurocomputing 86 (2012) 116–123.
[20] Mohsen Mohammadi, Faraz Talebpour, Esmaeil Safaee, Noradin Ghadimi, Oveis Abedinia, Small-scale building load forecast based on hybrid forecast engine, Neural Process. Lett. 48 (1) (2018) 329–351.
[21] Luis G. Baca Ruiz, R. Rueda, Manuel P. Cuéllar, M.C. Pegalajar, Energy consumption forecasting based on Elman neural networks with evolutive optimization, Expert Syst. Appl. 92 (2018) 380–389.
[22] Xiaolei Ma, Zhimin Tao, Yinhai Wang, Haiyang Yu, Yunpeng Wang, Long short-term memory neural network for traffic speed prediction using remote microwave sensor data, Transp. Res. C 54 (2015) 187–197.
[23] Yongxue Tian, Li Pan, Predicting short-term traffic flow by long short-term memory recurrent neural network, in: 2015 IEEE International Conference on Smart City/SocialCom/SustainCom (SmartCity), IEEE, 2015, pp. 153–158.
[24] Salah Bouktif, Ali Fiaz, Ali Ouni, Mohamed Adel Serhani, Optimal deep learning LSTM model for electric load forecasting using feature selection and genetic algorithm: Comparison with machine learning approaches, Energies 11 (7) (2018) 1636.
[25] Thomas Fischer, Christopher Krauss, Deep learning with long short-term memory networks for financial market predictions, European J. Oper. Res. 270 (2) (2018) 654–669.
[26] Kasun Bandara, Christoph Bergmeir, Slawek Smyl, Forecasting across time series databases using recurrent neural networks on groups of similar series: A clustering approach, Expert Syst. Appl. 140 (2020) 112896.
[27] Cheng Pan, Jie Tan, Dandan Feng, Yi Li, Very short-term solar generation forecasting based on LSTM with temporal attention mechanism, in: 2019 IEEE 5th International Conference on Computer and Communications, ICCC, IEEE, 2019, pp. 267–271.
[28] Alaa Sagheer, Mostafa Kotb, Time series forecasting of petroleum production using deep lstm recurrent networks, Neurocomputing 323 (2019) 203–213.
[29] Slawek Smyl, A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting, Int. J. Forecast. 36 (1) (2020) 75–85.
[30] Guo Yanan, Cao Xiaoqun, Liu Bainian, Peng Kecheng, Chaotic time series prediction using LSTM with CEEMDAN, in: Journal of Physics: Conference Series, vol. 1617, (1) IOP Publishing, 2020, 012094.
[31] Matteo Sangiorgio, Fabio Dercole, Robustness of LSTM neural networks for multi-step forecasting of chaotic time series, Chaos Solitons Fractals 139 (2020) 110045.
[32] Anastasia Borovykh, Sander Bohte, Cornelis W. Oosterlee, Dilated convolutional neural networks for time series forecasting, Journal of Computational Finance, Forthcoming (2018).
[33] Renzhuo Wan, Shuping Mei, Jun Wang, Min Liu, Fan Yang, Multivariate temporal convolutional network: A deep neural networks approach for multivariate time series forecasting, Electronics 8 (8) (2019) 876.
[34] Yitian Chen, Yanfei Kang, Yixiong Chen, Zizhuo Wang, Probabilistic forecasting with temporal convolutional neural network, Neurocomputing 399 (2020) 491–501.
[35] Pedro Lara-Benítez, Manuel Carranza-García, José M. Luna-Romera, José C. Riquelme, Temporal convolutional networks applied to energy-related time series forecasting, Appl. Sci. 10 (7) (2020) 2322.
[36] Wei Cheng, Yan Wang, Zheng Peng, Xiaodong Ren, Yubei Shuai, Shengyin Zang, Hao Liu, Hao Cheng, Jiagui Wu, High-efficiency chaotic time series prediction based on time convolution neural network, Chaos Solitons Fractals 152 (2021) 111304.
[37] Shaojie Bai, J. Zico Kolter, Vladlen Koltun, An empirical evaluation of generic convolutional and recurrent networks for sequence modeling, 2018, arXiv preprint arXiv:1803.01271.
[38] Colin Lea, Rene Vidal, Austin Reiter, Gregory D. Hager, Temporal convolutional networks: A unified approach to action segmentation, in: European Conference on Computer Vision, Springer, 2016, pp. 47–54.
[39] Sepp Hochreiter, Jürgen Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[40] Pedro Lara-Benítez, Manuel Carranza-García, José C. Riquelme, An experimental review on deep learning architectures for time series forecasting, Int. J. Neural Syst. 31 (03) (2021) 2130001.
[41] Keith W. Hipel, A. Ian McLeod, Time Series Modelling of Water Resources and Environmental Systems, Elsevier, 1994.
[42] Charles Elton, Mary Nicholson, The ten-year cycle in numbers of the lynx in Canada, J. Anim. Ecol. (1942) 215–244.
[43] Adamantios Ntakaris, Martin Magris, Juho Kanniainen, Moncef Gabbouj, Alexandros Iosifidis, Benchmark dataset for mid-price prediction of limit order book data, 2017, arXiv preprint arXiv:1705.03233.
[44] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, WaveNet: A generative model for raw audio, 2016, arXiv preprint arXiv:1609.03499.
[45] Aäron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew W. Senior, Koray Kavukcuoglu, WaveNet: A generative model for raw audio, SSW 125 (2016) 2.
[46] Hailun Xie, Li Zhang, Chee Peng Lim, Evolving CNN-LSTM models for time series prediction using enhanced grey wolf optimizer, IEEE Access 8 (2020) 161519–161541.
[47] Xinghan Xu, Weijie Ren, A hybrid model of stacked autoencoder and modified particle swarm optimization for multivariate chaotic time series forecasting, Appl. Soft Comput. 116 (2022) 108321.
[48] Jing Bi, Xiang Zhang, Haitao Yuan, Jia Zhang, MengChu Zhou, A hybrid prediction method for realistic network traffic with temporal convolutional network and lstm, IEEE Trans. Autom. Sci. Eng. (2021).
[49] George B. Moody, Roger G. Mark, The impact of the MIT-bih arrhythmia database, IEEE Eng. Med. Biol. Mag. 20 (3) (2001) 45–50.
[50] A. Goldberger, L. Amaral, L. Glass, J. Hausdorff, P.C. Ivanov, R. Mark, J.E. Mietus, G.B. Moody, C.K. Peng, H.E. Stanley, PhysioBank, PhysioToolkit, PhysioNet: components of a new research resource for complex physiologic signals, Circulation 101 (23) (2000) e215–e220.
[51] Hu Xiaoyan, Li Bingjie, Shi Jing, Li Hua, Liu Guojing, A novel forecasting method for short-term load based on TCN-gru model, in: 2021 IEEE International Conference on Energy Internet, ICEI, IEEE, 2021, pp. 79–83.
[52] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, Yoshua Bengio, Learning phrase representations using RNN encoder-decoder for statistical machine translation, 2014, arXiv preprint arXiv:1406.1078.
[53] Edward N. Lorenz, Deterministic nonperiodic flow, J. Atmos. Sci. 20 (2) (1963) 130–141.
[54] Alan Wolf, Jack B Swift, Harry L Swinney, John A Vastano, Determining Lyapunov exponents from a time series, Physica D 16 (3) (1985) 285–317.
[55] Otto E. Rössler, An equation for continuous chaos, Phys. Lett. A 57 (5) (1976) 397–398.
[56] Bao Bocheng, Liu Zhong, Xu Jianping, New chaotic system and its hyperchaos generation, J. Syst. Eng. Electronics 20 (6) (2009) 1179–1187.
[57] Varun Gupta, Monika Mittal, Vikas Mittal, Chaos theory: an emerging tool for arrhythmia detection, Sens. Imaging 21 (1) (2020) 1–22.
[58] Oleg Gorshkov, Hernando Ombao, Multi-chaotic analysis of inter-beat (RR) intervals in cardiac signals for discrimination between normal and pathological classes, Entropy 23 (1) (2021) 112.
[59] V. Gupta, M. Mittal, V. Mittal, R-peak detection using chaos analysis in standard and real time ECG databases, IRBM 40 (6) (2019) 341–354.
[60] Leon Glass,
[61] Varun Gupta, Monika Mittal, QRS complex detection using STFT, chaos analysis, and PCA in standard and real-time ECG databases, J. Inst. Eng. (India): Ser. B 100 (5) (2019) 489–497.
[62] Steven M. LaValle, Michael S. Branicky, Stephen R. Lindemann, On the relationship between classical grid search and probabilistic roadmaps, Int. J. Robot. Res. 23 (7–8) (2004) 673–692.
[63] Tianfeng Chai, Roland R. Draxler, Root mean square error (RMSE) or mean absolute error (MAE)?–Arguments against avoiding RMSE in the literature, Geosci. Model Dev. 7 (3) (2014) 1247–1250.
[64] Ummul Khair, Hasanul Fahmi, Sarudin Al Hakim, Robbi Rahim, Forecasting error calculation with mean absolute deviation and mean absolute percentage error, in: Journal of Physics: Conference Series, 930, (1) IOP Publishing, 2017, 012002.
[65] Gloria B. Barrett, The Coefficient of Determination: Understanding r squared and R squared, Math. Teach. 93 (3) (2000) 230–234.

[66] Milton Friedman, The use of ranks to avoid the assumption of normality implicit in the analysis of variance, J. Amer. Statist. Assoc. 32 (200) (1937) 675–701.
[67] Milton Friedman, A correction: The use of ranks to avoid the assumption of normality implicit in the analysis of variance, J. Amer. Statist. Assoc. 34 (205) (1939) 109.
[68] Peter Bjorn Nemenyi, Distribution-Free Multiple Comparisons., Princeton University, 1963.



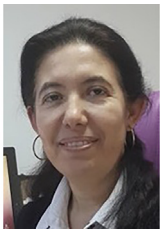
Hatice Vildan Dudukcu received B.Sc. (2015) and M.Sc. (2018) degrees in Electronics and Communication Engineering, from Yildiz Technical University (YTU), Istanbul, Turkey. She is currently a Ph.D. student at YTU. Since 2016, she has been working as a research assistant in the Department of Electronics and Communication Engineering in YTU. Her research interests are in signal processing, artificial neural networks, and biomedical instrumentation.



Murat Taskiran received B.Sc. (2013), M.Sc. (2016) and Ph.D. (2022) degrees in Electronics and Communication Engineering, from Yildiz Technical University (YTU), Istanbul, Turkey. From 2014 to 2021, he had been working as a research assistant in the Department of Electronics and Communication Engineering in YTU and he is currently working as a lecturer at YTU. His research interests are in image processing, neural networks, randomness analysis and time series prediction.



Zehra Gulru Cam Taskiran received the B.S. and M.Sc. degrees in Electronics Engineering from Istanbul Technical University, Istanbul, Turkey in 2011 and 2013, respectively. She received the Ph.D. degree in Electronics and Communications Engineering from Yildiz Technical University, Istanbul, Turkey, in 2018. Between 2013–2018, she was a research assistant and since Sep. 2018, she is an assistant professor in the Department of Electronics and Communications Engineering at Yildiz Technical University, Istanbul, Turkey. Dr. Cam Taskiran's current research interests are in the areas of circuit theory, nonlinear circuits, memristive circuits and chaotic circuits.



Tülay Yildirim received the B.Sc. and M.Sc. degrees in electronics and communication engineering from Yildiz Technical University, Istanbul, Turkey, in 1990 and 1992, respectively, and the Ph.D. degree in electrical and electronics engineering from the University of Liverpool, U.K., in 1997. She is currently a Full Professor with the Department of Electronics and Communications Engineering, Yildiz Technical University. She has authored over 200 publications in journals, conferences, and books. Her research interests include artificial intelligence, intelligent systems, cyber–physical systems, biomedical instrumentation, biometric person identification, and verification systems, electronic circuits and systems, and artificial neural networks.